Academic Year: 2026-2027
Semester: III
Class / Branch: SE IT
Subject:ADSAL lab
Name of Instructor:prof.Sneha Dalvi

Name of Student:Nishant Suryarao
Student ID:26204001
Date of Performance:06/10/2026
Date of Submission:06/10/2026

Experiment No.7

Aim: Implement graph colouring.

Code:

```
#include <stdio.h>

#define MAX 10

int V;
int graph[MAX][MAX];
int color[MAX];

// Check if vertex v can be assigned color c
int isSafe(int v, int c)
{
   for (int i = 0; i < V; i++)
   {
      if (graph[v][i] && color[i] == c)
         return 0;
   }

   return 1;
}

// Backtracking function
int graphColoring(int v, int m)
{
   // All vertices are colored
   if (v == V)
      return 1;
```

```
    // Try each color
    for (int c = 1; c <= m; c++)
    {
        if (isSafe(v, c))
        {
            color[v] = c;

            // Recursively color the next vertex
            if (graphColoring(v + 1, m))
                return 1;

            // Backtrack
            color[v] = 0;
        }
    }

    return 0;
}

int main()
{
    int m;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    if (V > MAX || V <= 0)
    {
        printf("Invalid number of vertices. Maximum is %d.\n", MAX);
        return 1;
    }

    printf("Enter adjacency matrix (%d x %d):\n", V, V);

    for (int i = 0; i < V; i++)
    {
        for (int j = 0; j < V; j++)
        {
            scanf("%d", &graph[i][j]);
        }
```

```
    }

    printf("Enter number of colors: ");
    scanf("%d", &m);

    // Initialize all vertices as uncolored
    for (int i = 0; i < V; i++)
        color[i] = 0;

    // Solve graph coloring
    if (graphColoring(0, m))
    {
        printf("\nColor Assignment:\n");

        for (int i = 0; i < V; i++)
        {
            printf("Vertex %d ---> Color %d\n", i, color[i]);
        }
    }
    else
    {
        printf("\nSolution does not exist with %d colors.\n", m);
    }

    return 0;
}
```

Output Screen Shots:

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ gcc -o Exp7 Exp7.c
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ ./Exp7
Enter number of vertices: 3
Enter adjacency matrix (3 x 3):
5 6 4
9 8 7
3 5 4
Enter number of colors: 3

Color Assignment:
Vertex 0 ---> Color 1
Vertex 1 ---> Color 2
Vertex 2 ---> Color 3
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$
```

Conclusion : Graph Coloring is a fundamental NP-Complete problem that demonstrates the balance between theoretical complexity and practical utility. While the backtracking algorithm provides exact solutions, its exponential time complexity limits scalability to small graphs. Despite computational challenges, Graph Coloring remains crucial in real-world applications like compiler optimization, network channel assignment, and scheduling systems.

Conclusion: